PiBlock

Servidor de Minecraft portable per a entorns educatius

Memòria Tècnica del Projecte Intermodular

Mohamed Amin Adaardour · Diego Morazán
INS Palamós · SMX2 · Mòdul M12
Tutor: Antonio Alcalà
Maig 2026

Índex

Índex **2**
1. Resum executiu **3**
1.1 Català 3
1.2 English (Abstract) 3
2. Context i situació inicial **3**
2.1 El problema 3
2.2 La proposta PiBlock 4
2.3 Estat actual del projecte 4
3. Abast del projecte **4**
3.1 Què inclou i què no 4
3.2 Actors del sistema 5
4. Casos d'ús **5**
CU-01: Modificar la configuració del servidor 5
CU-02: Gestionar les còpies de seguretat 6
CU-03: Registrar una nova màquina PiBlock 7
CU-04: Consultar l'estat dels servidors de l'entitat 7
CU-05: Connectar-se al servidor de Minecraft 8
CU-06: Consultar la informació del servidor 8
CU-07: Configurar el món per a una activitat educativa 9
CU-08: Supervisar l'activitat dels alumnes 9
5. Coneixements del cicle aplicats **10**
6. Disseny i solució tècnica **11**
6.1 Taula creuada Capes × Casos d'ús 11
Casos de l'administrador i del client 11
Casos del jugador i del professor 12
6.2 Flux d'una connexió 12
6.3 Arquitectura física 12
6.4 Descripció de les capes 13
Aplicacions 13
Serveis 13
Sistema Operatiu 13
Xarxa i Comunicacions 13
Capa Física 14
6.5 Justificació de les decisions tècniques 14
6.6 Aspectes transversals 14
Estàtics 14
Dinàmics 14
7. Planificació **15**
7.1 Fases i fites 15
7.2 Tasques amb Definition of Done 15
T1: Preparació del sistema operatiu (Capa SO + Física) 15
T2: Configuració del túnel d'accés extern (Capa Xarxa) 15

T3: Panell de gestió (Aplicacions + Serveis) 16
T4: Stack de Minecraft (Capa Serveis) 16
T5: Web PHP (Aplicacions, tasca obligatòria M12) 16
T6: Documentació final 16
7.3 Diagrama de Gantt 17
7.4 Rols de l'equip 17
8. Desenvolupament i implementació 17
8.1 Estructura del repositori 17
8.2 Bones pràctiques Git 17
8.3 Tasca obligatòria: web PHP 17
Esquema de la base de dades 18
Fragment: autenticació segura 18
8.4 Tasques opcionals completades 18
9. Desplegament 18
9.1 Arquitectura de desplegament 18
9.2 Passos del desplegament 18
9.3 Incidències trobades 18
9.4 Validació final 19
10. Resultats i validació 19
10.1 Objectius assolits 19
10.2 Proves realitzades 19
10.3 Feedback i validació externa 20
11. Conclusions i aprenentatges 20
11.1 El que ha funcionat 20
11.2 El que ha costat més 20
11.3 Millores futures 20
11.4 Reflexió sobre els mòduls 21
12. Annexos 21
Annex A. Repositori i recursos 21
Annex B. Fragment de la configuració de Velocity 21
Annex C. Captures de pantalla 21
Annex D. Scripts d'instal·lació 21

1. Resum executiu

1.1 Català

Muntar un servidor de Minecraft a una escola és més complicat del que sembla. Cal saber d'administració de sistemes, instal·lar el joc, configurar versions, fer còpies, mantenir-ho. La majoria de centres no tenen ningú que ho pugui o vulgui fer, i la idea de fer servir Minecraft com a eina educativa s'acaba abandonant.

PiBlock soluciona això amb un dispositiu físic que arriba pre-configurat. L'escola el connecta a la xarxa i el sistema ja funciona. La gestió del servidor es fa de manera remota per web. Els jugadors es connecten tant si fan servir Minecraft Java com Bedrock, i poden compartir el mateix món.

L'accés des de fora no necessita obrir cap port al tallafoc del centre. Tot el tràfic va per un túnel xifrat. El sistema cobreix vuit casos d'ús repartits entre quatre actors: l'administrador (gestiona el servidor), l'entitat client (registra i supervisa les seves màquines), el jugador (es connecta a jugar) i el professor (configura activitats i supervisa els alumnes).

1.2 English

Setting up a Minecraft server at a school is more complicated than it seems. It requires systems administration knowledge, installing the game, configuring versions, creating backups, and maintaining everything. Most schools do not have anyone willing or able to do it, and the idea of using Minecraft as an educational tool ends up being abandoned.

PiBlock solves this with a physical device that comes preconfigured. The school connects it to the network and the system is ready to use. Server management is done remotely through a web interface. Players can connect whether they use Minecraft Java Edition or Bedrock Edition, and they can share the same world.

Access from outside does not require opening any ports on the school firewall. All traffic goes through an encrypted tunnel. The system covers eight use cases distributed across four actors: the administrator (manages the server), the client organization (registers and monitors its devices), the player (connects to play), and the teacher (configures activities and supervises students).

2. Context i situació inicial

2.1 El problema

Minecraft té anys de recorregut com a eina d'aula. No tant pel joc en si, sinó perquè és un entorn que els alumnes ja coneixen i que serveix per estudiar diversos temes com ara geometria, programació bàsica, treball en equip, resolució de problemes, entre altres. El que falla sempre és la part del servidor.

Un servidor de Minecraft no és tan complicat si saps el que fas. El problema és que als centres normalment no hi ha ningú que ho sàpiga, o que vulgui dedicar-hi hores. Les opcions habituals són: que un alumne llest el munti (i ningú sap què passa quan marxa), pagar un hosting extern (car, sense control del maquinari, sovint amb latència alta) o deixar-ho córrer.

Hi ha també el tema de versions. Minecraft es ven en dues variants que no parlen entre elles: la Java, la dels ordinadors, i la Bedrock, la dels mòbils i les consoles. Sense una solució específica, els alumnes amb un dispositiu de l'altra versió no es poden connectar al mateix món.

I per acabar-ho de complicar, l'accés des de fora. Si vols que els alumnes es connectin des de casa cal que el servidor sigui visible des d'internet. Això vol dir obrir ports al router, una cosa que els departaments de xarxes acostumen a prohibir.

2.2 La proposta PiBlock

La idea és deixar tots aquests obstacles fora del camí. L'escola rep un dispositiu físic ja preparat. No cal saber res de servidors. No cal tocar la xarxa. Endolles el dispositiu, el registres al sistema i ja està.

A partir d'aquí, l'administrador (un professor o qui designi el centre) gestiona el servidor de manera remota. Pot canviar la configuració del joc, afegir o treure jugadors, fer còpies, reiniciar el servidor si cal. Tot des de la web, sense fitxers de configuració, sense Linux. El professor pot configurar el món per a una activitat concreta i supervisar els alumnes.

Els jugadors Java i Bedrock es connecten directament. Si el servidor de joc cau, no es desconnecten del tot: van a parar a una sala d'espera i, quan el servidor torna, hi entren automàticament.

2.3 Estat actual del projecte

A data d'aquesta memòria tenim operativa la part central de la infraestructura. El servidor de joc funciona i accepta clients de les dues versions. L'accés extern xifrat funciona sense haver d'obrir ports. La gestió remota permet veure l'estat del servidor i administrar-lo.

Queda pendent acabar: les còpies de seguretat programades, la integració del registre automàtic de noves màquines i algunes parts de la gestió remota (llista d'autoritzats, registres en temps real).

3. Abast del projecte

3.1 Què inclou i què no

Component / servei	Inclou	No inclou
Màquina servidora del joc	✓	
Sistema central de registre i gestió de comptes	✓	
Configuració del joc preestablerta	✓	
Còpies de seguretat manuals	✓	
Compatibilitat Java i Bedrock	✓	
Accés extern segur sense obrir ports	✓	
Gestió remota del servidor	✓	
Llicències del joc		✗
Dispositius dels jugadors		✗
Connexió a internet del client		✗
Subministrament elèctric		✗

3.2 Actors del sistema

Hi ha quatre tipus d'usuaris amb necessitats i nivells d'accés diferents:

Actor	Rol
Administrador	**Gestiona tècnicament el servidor. Configuració del joc, jugadors, còpies, monitoratge i reinicis. Hi accedeix de manera remota.**
Client	**L'escola o organització. Rep el dispositiu, el registra i supervisa l'estat dels seus servidors. No toca configuració tècnica.**
Jugador	**Alumne o usuari que es connecta a jugar. Pot fer servir Java (PC) o Bedrock (mòbil, consola, PC). S'autentica amb el seu compte de Mojang/Microsoft o de Bedrock.**
Professor	**Docent que fa servir el servidor a classe. Configura el món per a una activitat i supervisa qui hi és i què fa. Pot expulsar alumnes que no segueixin la classe.**

4. Casos d'ús

Els casos d'ús descriuen què fan els actors amb el sistema, no com el sistema ho resol per dins. Les solucions tècniques van a la secció 6. N'hi ha vuit, dos per cada actor.

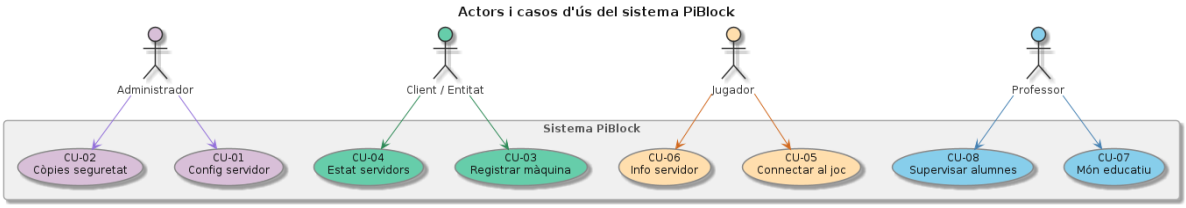


Figura 1. Actors i casos d'ús del sistema PiBlock.

CU-01: Modificar la configuració del servidor

L'administrador canvia paràmetres del joc (dificultat, mode, llista blanca, límit de jugadors) de manera remota.

Camp	Detall
Actor principal	**Administrador**
Actors secundaris	**Cap**
Precondicions	1) La màquina PiBlock és encesa i connectada a internet. 2) L'administrador té sessió iniciada al panell. 3) El servidor de joc funciona.
Flux principal	1) Va a la secció de configuració del panell. 2) El sistema mostra els valors actuals. 3) Modifica el que vol i confirma. 4) El sistema valida, aplica el canvi i confirma. 5) Els jugadors connectats noten el canvi al moment o a la proper reinici.
Flux alternatiu	Valor fora de rang: rebutjat amb missatge o no funciona. Màquina no connectada: el panell ho indica i permet reintentar.
Postcondicions	Configuració actualitzada. Canvi al registre d'activitat.
Requisits especials	Canvis aplicats en menys de 5 segons. Disponibilitat ≥ 99%.

CU-02: Gestionar les còpies de seguretat

L'administrador protegeix el món davant pèrdues. Pot crear còpies manuals i restaurar-ne.

Camp	Detall
Actor principal	**Administrador**
Actors secundaris	**Cap**
Precondicions	1) Sessió iniciada al panell. 2) Espai d'emmagatzematge disponible.
Flux principal	1) Va a la secció de còpies. 2) Veu la llista (data, hora, mida). 3) Per crear: prem "Còpia manual", el sistema la fa en segon pla. 4) Per restaurar: tria una còpia, confirma. El sistema avisa que el servidor s'aturarà breument. 5) Restaura, reinicia el servidor, confirma.
Flux alternatiu	Sense espai: el sistema avisa abans de començar. Restauració falla: cancel·la i torna a l'estat anterior. Jugadors connectats: són desconnectats de manera ordenada.
Postcondicions	Nova còpia disponible o món restaurat al punt triat.
Requisits especials	Verificació d'integritat de cada còpia. Procés en calent: no atura el joc.

CU-03: Registrar una nova màquina PiBlock

L'entitat rep el dispositiu i el vincula al seu compte per obtenir credencials de gestió.

Camp	Detall
Actor principal	**Client / Entitat**
Actors secundaris	**Administrador (rep les credencials)**
Precondicions	1) Compte actiu al sistema. 2) Dispositiu rebut i connectat a internet.
Flux principal	1) Inicia sessió al panell de l'entitat. 2) Veu les seves màquines (si n'hi ha). 3) Tria "afegir màquina" i introdueix l'identificador imprès a la caixa. 4) El sistema verifica que existeix i no està registrada. 5) Vincula la màquina al compte i genera credencials d'admin. 6) Mostra les credencials per anotar-les i passar-les a l'admin.
Flux alternatiu	Identificador invàlid: avís. Ja registrat en un altre compte: el sistema dóna un canal de contacte. Sense connexió: el registre es completa però la màquina no s'ha pogut verificar online.
Postcondicions	Màquina visible al panell de l'entitat. Admin amb credencials vàlides.
Requisits especials	Credencials mostrades una sola vegada (no es desen en clar). Cada màquina només pot estar a un compte.

CU-04: Consultar l'estat dels servidors de l'entitat

L'entitat vol saber quants servidors té actius, quants jugadors hi ha i si algun té problemes.

Camp	Detall
Actor principal	**Client / Entitat**
Actors secundaris	**Cap**
Precondicions	1) Compte actiu. 2) Almenys una màquina registrada.
Flux principal	1) Inicia sessió al panell. 2) Veu la llista: nom, estat, jugadors connectats. 3) Pot obrir un servidor concret per veure data de registre, última connexió i contacte de suport.
Flux alternatiu	Cap màquina: el sistema ho diu i ofereix registrar-ne una. Servidor sense connexió: indica l'última activitat.
Postcondicions	Visió actualitzada de l'estat dels servidors.
Requisits especials	Cada entitat només veu les seves dades. Retard màxim de 2 minuts respecte a la realitat.

CU-05: Connectar-se al servidor de Minecraft

Un alumne entra a jugar amb la versió Java o Bedrock. Si el servidor cau, va a la sala d'espera i hi torna automàticament.

Camp	Detall
Actor principal	**Jugador**
Actors secundaris	**Cap**
Precondicions	1) El servidor és accessible des d'internet. 2) El jugador té el client (Java o Bedrock) i un compte actiu. 3) Coneix l'adreça del servidor.
Flux principal	1) Obre el client i s'identifica. 2) Afegeix el servidor. 3) Es connecta. El sistema valida el compte i comprova si té accés. 4) Si és Bedrock, Geyser tradueix els paquets en directe. 5) Entra al món i pot jugar amb la resta, sigui quina sigui la versió.
Flux alternatiu	Servidor caigut: el jugador va a la sala d'espera, hi torna sol quan reviu (límit configurable, per defecte 10 min). Sense permís: missatge informatiu. Caiguda durant la sessió: redirigit a la sala d'espera en menys de 5 segons, sense missatge brusc.
Postcondicions	Jugador al món, autenticat. Si ha passat per la sala d'espera, hi torna sol.
Requisits especials	Latència < 50 ms local, < 120 ms extern. Bedrock: traducció afegeix < 20 ms. Compatibilitat amb les versions de Bedrock dels darrers 12 mesos.

CU-06: Consultar la informació del servidor

Abans d'entrar, el jugador vol saber si el servidor és en línia, quants jugadors hi ha i si la seva versió és compatible.

Camp	Detall
Actor principal	**Jugador**
Actors secundaris	**Cap**
Precondicions	1) Coneix l'adreça. 2) Té internet.
Flux principal	1) Afegeix el servidor a la llista del client. 2) El client envia un ping. 3) El sistema respon amb estat, nom, jugadors, màxim i versió. 4) El jugador decideix si entrar.
Flux alternatiu	Sense resposta: client mostra "fora de línia". Versió incompatible: missatge específic.
Postcondicions	Jugador amb informació actualitzada.
Requisits especials	Resposta del ping en menys de 500 ms. Dades de jugadors amb 30 s de retard màxim.

CU-07: Configurar el món per a una activitat educativa

El professor prepara el món per a una activitat: llavor, mode de joc, fronteres, o restablir-lo a un estat conegut.

Camp	Detall
Actor principal	**Professor**
Actors secundaris	**Jugadors**
Precondicions	1) Credencials amb rol de Professor. 2) Servidor en marxa.
Flux principal	1) Va a la secció de configuració educativa. 2) Tria l'acció (llavor, mode, fronteres, restablir). 3) Configura paràmetres i confirma. 4) El sistema aplica. Si cal reiniciar (canvi de llavor), avisa els jugadors i ho fa.
Flux alternatiu	Cap estat guardat: ofereix crear-ne un de nou. Jugadors connectats i acció amb reinici: demana confirmació.
Postcondicions	Món configurat per a l'activitat. Canvi al registre.
Requisits especials	Sense coneixements tècnics de Minecraft. Tot canvi reversible. Mode de joc i fronteres no requereixen reinici.

CU-08: Supervisar l'activitat dels alumnes

Durant la classe, el professor vol veure qui hi és, quant temps fa que hi és i poder expulsar algú que no segueix la classe.

Camp	Detall
Actor principal	**Professor**
Actors secundaris	**Jugadors**
Precondicions	1) Sessió iniciada amb rol de Professor. 2) Servidor en marxa.
Flux principal	1) Va a la secció de supervisió. 2) Veu la llista: nom, versió, temps de connexió. 3) Pot obrir un jugador per veure posició al món i última acció. 4) Si cal, expulsa algú; el sistema el desconnecta i li mostra un missatge.
Flux alternatiu	Cap jugador: avís. Expulsió d'algú ja desconnectat: el sistema ho indica.
Postcondicions	Professor amb visibilitat de l'activitat. Jugadors expulsats no entren fins que el professor o l'admin ho permetin.
Requisits especials	Privacitat: només dades necessàries per a la classe. Llista refrescada cada 30 s com a màxim. Expulsions amb temps configurable.

5. Coneixements del cicle aplicats

Cada mòdul del cicle SMX2 té un paper al projecte. Els que han pesat més són M07 (Serveis de Xarxa), M09 (Seguretat), M10 (Aplicacions Web) i M01 (Sistemes Operatius). El detall complet és a la taula següent.

Mòdul	On s'ha aplicat
M07 — Serveis de Xarxa i Internet	**Stack de Minecraft: Paper, Velocity, Geyser, Limbo (executats com a serveis systemd). Pyrodactyl per a la gestió de servidors addicionals (Docker via Elytra).**
M09 — Seguretat Informàtica	**Playit.gg sense obrir ports. Autenticació. Aïllament Docker. Protecció contra atacs comuns.**
M10 — Aplicacions Web	**Web PHP de gestió. Base de dades MariaDB. Integració amb l'API via cURL. Sessions i autenticació.**
M01 — Sistemes Operatius	**Ubuntu Server al Pi5. Particionat, usuaris, permisos. Hardening (SSH amb clau, firewall, actualitzacions).**
M02 — Maquinari	**Selecció del Pi5. Compatibilitat ARM amb Ubuntu. Emmagatzematge.**
M03 — Projecte Intermodular	**ClickUp, fases, fites. Repartició de feines. Documentació al repo.**
M04 — Llenguatges de marques	**HTML i CSS de la web. JSON per a la comunicació amb l'API.**
M05 — Seguretat de Xarxes	**Tallafoc del sistema. Segmentació entre contenidors. Polítiques del túnel.**
M06 — Xarxes Locals	**Adreçament IP, DHCP, DNS. Configuració del túnel.**
M08 — Administració de SO	**Docker, systemd, scripts bash. Còpies. Monitoratge.**

6. Disseny i solució tècnica

Un cop definits els casos d'ús, la pregunta és: quins components fan falta? Amb la descomposició en cinc capes (Aplicacions, Serveis, Sistema Operatiu, Xarxa, Física), la resposta surt sola.

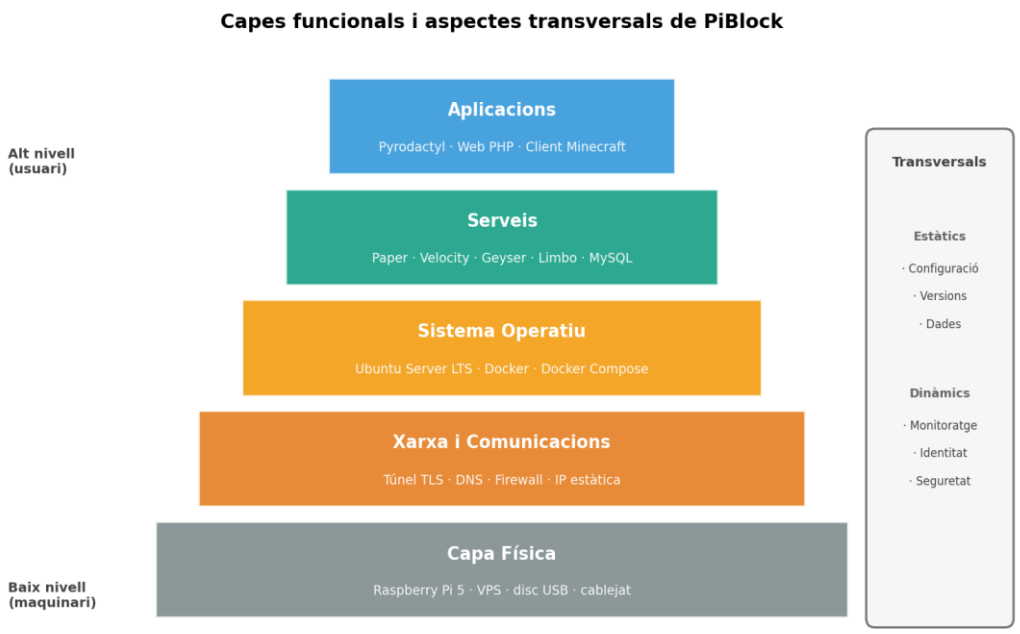


Figura 2. Capes funcionals de PiBlock i aspectes transversals.

6.1 Taula creuada Capes × Casos d'ús

Cada cel·la diu quin component d'aquella capa dóna suport a aquell cas d'ús. Els components es repeteixen a les capes baixes (és normal, el sistema operatiu i la xarxa sostenen tot el sistema alhora).

Casos de l'administrador i del client

Capa	CU-01 Config	CU-02 Còpies	CU-03 Registre	CU-04 Estat ent.
Aplicacions	Web de gestió	Web de gestió	Form. + auto-reg.	Panell d'entitat
Serveis	API panell	API + sistema fitxers	API + auto-reg.	API d'estat
Sist. Operatiu	Linux + Docker	Linux + cron	Linux + Docker	Linux + Docker
Xarxa	Playit.gg	Playit.gg	DNS + túnel	DNS + túnel
Física	Pi 5	Pi 5 + disc	Pi 5	Pi 5

Casos del jugador i del professor

Capa	CU-05 Connexió	CU-06 Info	CU-07 Món edu.	CU-08 Supervisió
Aplicacions	Client Minecraft	Client Minecraft	Interfície educ.	Interfície educ.
Serveis	Paper + Velocity	Ping de servidor	API + gestió móns	API + jugadors
Sist. Operatiu	Linux + Docker	Linux + Docker	Linux + Docker	Linux + Docker
Xarxa	Túnel + DNS	Túnel + DNS	Playit.gg	Playit.gg
Física	Pi 5	Pi 5	Pi 5	Pi 5

6.2 Flux d'una connexió

Val la pena seguir el camí d'una connexió des de fora del centre fins que el jugador apareix al món:

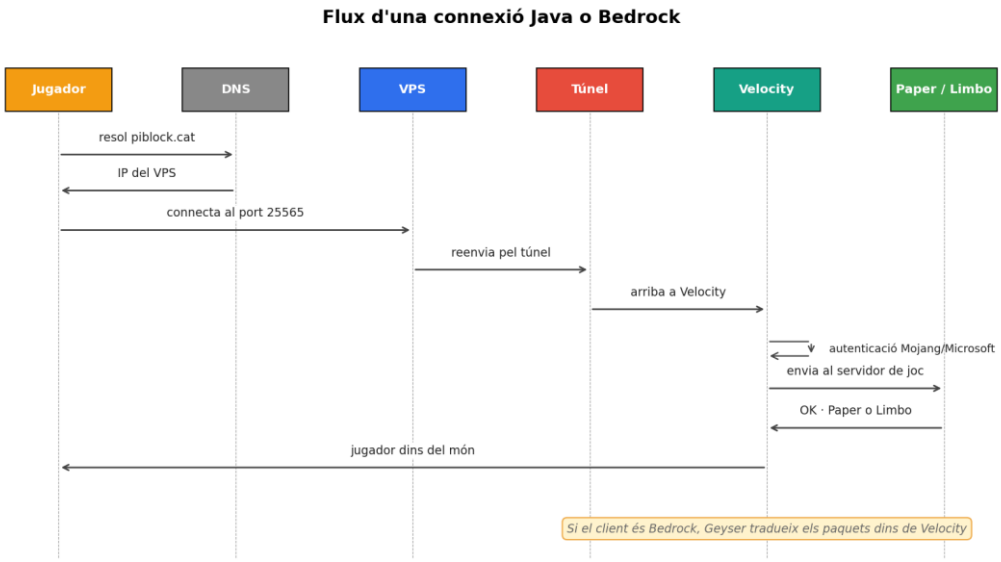


Figura 3. Seqüència d'una connexió Java o Bedrock.

El client resol piblock.cat al DNS i obté l'endpoint públic del túnel Playit.gg. La connexió travessa el túnel xifrat (sortint des del Pi 5, sense ports oberts al router del centre) i arriba a Velocity al port 25565. Velocity autentica el jugador Java contra els servidors de Mojang/Microsoft; si el client és Bedrock, Geyser tradueix els paquets i Floodgate gestiona l'autenticació amb la llicència de Bedrock sense exigir una llicència de Java separada. Velocity envia el jugador a Paper. Si Paper no respon, va a parar a Limbo, la sala d'espera, fins que el servidor torna.

L'administrador i el professor accedeixen a la web de gestió per HTTPS. A la xarxa local entren directament al Pi 5; des de fora, la connexió també arriba pel túnel Playit.gg. La web parla amb

l'API de Pyrodactyl, que corre a la mateixa Pi i comanda el daemon que actua sobre els contenidors.

6.3 Arquitectura física

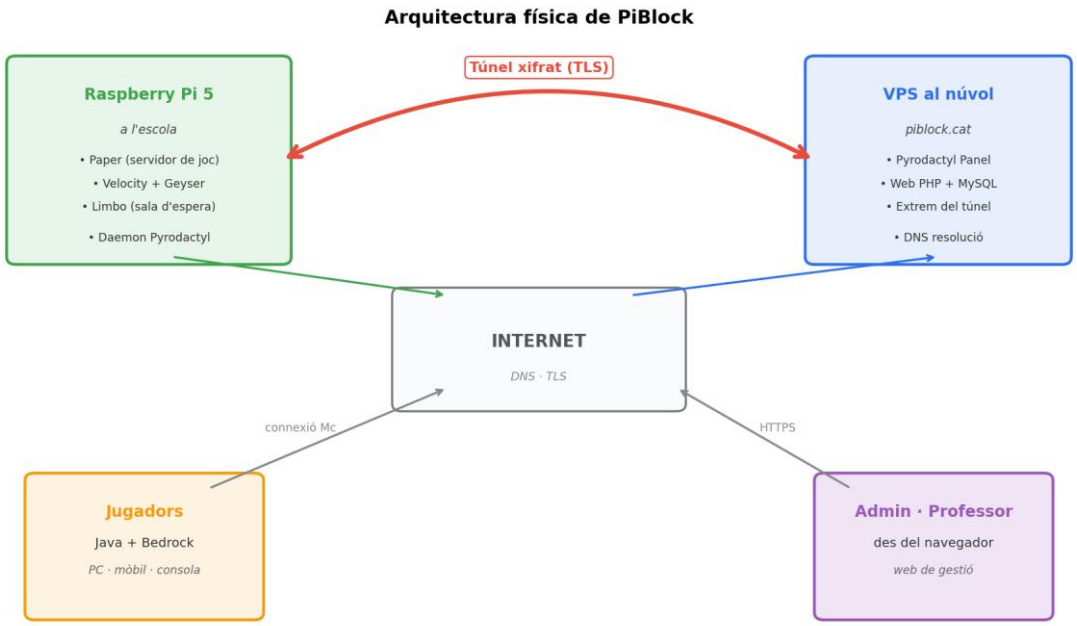


Figura 4. Arquitectura física: Raspberry Pi 5 autònoma a l'escola, amb túnel Playit.gg de sortida per a l'accés extern.

6.4 Descripció de les capes

Aplicacions

Pyrodactyl és el panell de gestió. Fa servir Docker per aïllar cada instància i exposa una API que rep ordres de la web i les aplica al servidor. Hi treballen l'administrador (CU-01, CU-02), el client/entitat (CU-03, CU-04) i el professor (CU-07, CU-08). La web PHP addicional (tasca obligatòria del M12) és l'accés simplificat per a l'admin i el professor.

El registre automàtic de noves màquines (CU-03) està pendent d'integrar amb Elytra. Els jugadors (CU-05, CU-06) entren directament amb el client de Minecraft.

Serveis

Paper és el servidor de joc, una versió optimitzada de Minecraft Java Edition. No és accessible directament des de fora.

Velocity és el proxy. Rep totes les connexions (Java i Bedrock), autentica i decideix on les envia. Si Paper cau, redirigeix els jugadors a la sala d'espera (CU-05, flux alternatiu).

Limbo és la sala d'espera. Pesa al voltant de 30 MB de RAM i sempre és viu, així el servidor principal pot estar en manteniment sense que els jugadors notin un tall brusc.

Geyser corre dins de Velocity i tradueix els paquets de Bedrock al format Java en directe. Permet als jugadors de mòbil i consola entrar al mateix món sense configuració extra.

Sistema Operatiu

Tot corre sobre Ubuntu Server amb Java 21 (OpenJDK). Els quatre serveis Minecraft (Paper, Velocity, Geyser, Limbo) corren com a unitats systemd independents (piblock-paper.service, piblock-velocity.service, etc.), cadascuna amb la seva política de reinici (Restart=on-failure). Docker s'instal·la addicionalment per al daemon Elytra de Pyrodactyl, que gestiona en contenidors aïllats els servidors addicionals creats des del panell.

Xarxa i Comunicacions

L'accés extern sense obrir ports es resol amb un túnel xifrat de sortida cap a Playit.gg. La Pi obre la connexió cap a l'endpoint públic, així el router del centre queda tancat (cap port forwarding, cap canvi al tallafoc). Des de fora, la màquina local segueix sent invisible. La resolució DNS de piblock.cat apunta a l'endpoint públic del túnel.

Capa Física

El maquinari és una Raspberry Pi 5 amb 8 GB de RAM. Aguanta una aula sencera (al voltant de 30 jugadors simultanis amb clients Java i Bedrock barrejats). El consum és de 5-8 W i la mida és la d'una capseta de mistos. L'emmagatzematge va sobre un SSD USB que conté móns, configuracions i còpies.

6.5 Justificació de les decisions tècniques

Component	Per què	Alternativa descartada
Paper	**Millor rendiment que Vanilla. Comunitat activa.**	**Vanilla: més consum, sense optimitzacions.**
Velocity	**Modern, plugins moderns, compatible amb Geyser nativament.**	**BungeeCord: obsolet, problemes de seguretat.**
Geyser	**Únic projecte madur per a Bedrock-a-Java. Lliure.**	**No hi ha alternativa equivalent lliure.**
Limbo	**Menys de 50 MB de RAM. Manté els jugadors mentre Paper reinicia.**	**Lobby de Velocity: més pesat, vol un món sencer.**
Raspberry Pi 5	**100€ una sola vegada, consum mínim, prou per a una aula sencera, fàcil d'enviar.**	**Mini PC x86: 200€+, més consum.**
Playit.gg	**Túnel de sortida (outbound only): cap canvi al router del centre.**	**Port-forwarding directe: prohibit a la majoria de centres.**
Pyrodactyl	**Panell especialitzat. API completa. Gestiona Docker sol.**	**Pterodactyl: més pesat per a maquinari limitat.**
PHP / MariaDB	**Requisit del M12. Suport ampli. Integració senzilla via cURL.**	**Node.js: més modern però no compleix el requisit formatiu.**

6.6 Aspectes transversals

Estàtics

Configuració: la web valida tot abans d'enviar. Els fitxers de Paper, Velocity, Geyser i Limbo viuen al directori config/ del repositori Git i es descarreguen automàticament a cada instal·lació. L'script install.sh fixa les versions exactes de cada component via URLs concretes. Tot canvi queda registrat al sistema de logs i és reversible.

Versions: l'install.sh fixa les versions concretes de Paper, Velocity, Limbo i tots els plugins (Floodgate, Geyser, Hurricane, PacketEvents, GeyserExtras) mitjançant URLs específiques. PHP, MariaDB i Nginx vénen dels paquets oficials de Debian/Ubuntu.

Protecció de dades: contrasenyes amb bcrypt a MariaDB. Còpies de seguretat del món per recuperar pèrdues. Playit.gg per al tràfic. Contenidors Docker aïllats. Disc extern per a les còpies. LOPD aplicada als noms d'usuari.

Dinàmics

Monitoratge: la web mostra estat i registres. Pyrodactyl monitora CPU, RAM i jugadors en temps real. El daemon mira els contenidors i el sistema de fitxers. Latència del túnel monitoritzada. La RPi té sensors de temperatura accessibles per comanda. El sistema avisa quan algun valor passa el llindar.

Identitat i accessos: l'admin i el professor s'autentiquen a la web. Els jugadors s'autentiquen contra Mojang/Microsoft sense doble login. Velocity valida sense afegir un segon pas. Contenidors amb usuaris separats i permisos mínims. Túnel amb autenticació bilateral.

Ciberseguretat: PDO amb prepared statements (anti-SQLi), escapament de sortida (anti-XSS), contrasenyes amb password_hash (bcrypt). Pyrodactyl, a través del seu daemon Elytra, aïlla els servidors addicionals en contenidors Docker amb usuari de sistema dedicat (pyrodactyl, UID 8888). Firewall UFW amb només els ports necessaris (22, 80, 443, 25565/TCP, 19132/UDP, 8080). Paper i Limbo no s'exposen al firewall. SSH amb clau pública.

7. Planificació

7.1 Fases i fites

Fase	Fita	Tasques principals
F1 — Inici	**M1: Sistema definit**	**Context, abast, casos d'ús, diagrama de flux.**
F2 — Planificació	**M2: Planificació tancada**	**Descomposició en capes. Taula creuada. Diagrama de Gantt.**
F3 — Execució	**M3: Execució**	**Scripts d'instal·lació, configuració de serveis, web PHP i proves al laboratori.**

F4 — Tancament	**M4: Lliurament final**	**Memòria final, presentació i defensa.**

7.2 Tasques amb Definition of Done

Les tasques surten de la taula creuada de la secció 6.1: cada tasca agrupa les cel·les del mateix component. Per a cadascuna definim el resultat, els criteris d'acceptació i els recursos.

T1: Preparació del sistema operatiu (Capa SO + Física)

Pi 5 amb Ubuntu Server actualitzat, accés SSH i Docker operatiu.

Definition of Done:

- SSH accessible des de la xarxa local.
- SO actualitzat a la darrera versió estable.
- Firewall configurat (només ports necessaris).
- Usuari admin amb clau pública.
- Docker i Docker Compose instal·lats.

Recursos: 1 persona, 2 hores

Casos d'ús coberts: Tots (SO i Física són transversals).

T2: Configuració del túnel d'accés extern (Capa Xarxa)

Servidor accessible des de fora sense obrir ports, amb tràfic xifrat.

Definition of Done:

- Connexió des d'IP externa funciona.
- Tràfic xifrat client ↔ servidor.
- Latència < 120 ms des de fora.
- Configuració documentada al repo.

Recursos: 1 persona, 3 hores

Casos d'ús coberts: CU-01 a CU-05 (Xarxa transversal).

T3: Panell de gestió (Aplicacions + Serveis)

Pyrodactyl operatiu i accessible per web.

Definition of Done:

- Panell web accessible.
- Daemon Elytra comunicant-se amb el panell Pyrodactyl.
- Serveis systemd piblock-* en marxa.
- Usuari admin i rol de professor creats.

Recursos: 1 persona, 4 hores

Casos d'ús coberts: CU-01, CU-02, CU-03, CU-04, CU-07, CU-08

T4: Stack de Minecraft (Capa Serveis)

Servidor de joc, proxy, traducció i sala d'espera connectats.

Definition of Done:

- Java connecta i juga.
- Bedrock connecta i juga.
- Failover a Limbo funciona quan Paper cau.
- Java i Bedrock es veuen al mateix món.
- Ping del servidor torna informació correcta (CU-06).
- Configuracions versionades al repo.

Recursos: 1 persona, 6 hores

Casos d'ús coberts: CU-05, CU-06

T5: Web PHP (Aplicacions, tasca obligatòria M12)

Web amb autenticació, estat i canvi de configuració. BD MariaDB.

Definition of Done:

- Login amb PDO + prepared statements.
- Estat del servidor visible (cURL a l'API).
- Canvi de dificultat i mode des de la web.
- Protecció anti-SQLi.
- Codi al directori web/.

Recursos: 1 persona, 8 hores

Casos d'ús coberts: CU-01, CU-04 (tasca obligatòria M12).

T6: Documentació final

Memòria, presentació web i repositori complets.

Definition of Done:

- Memòria PDF a docs/.
- Presentació accessible per URL.
- Estructura del repo correcta.
- Configuracions comentades.

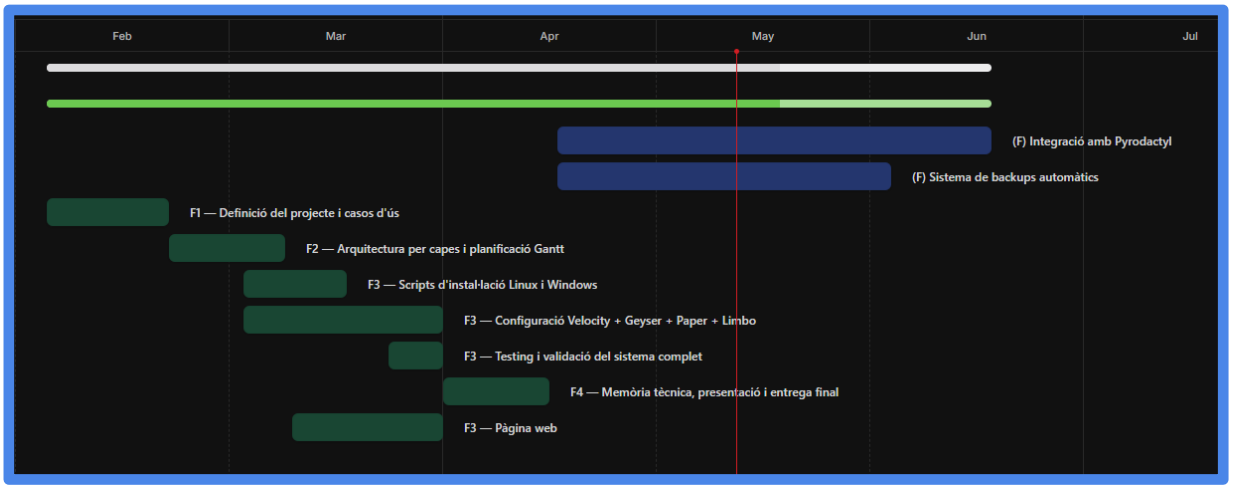
Recursos: 2 persones, 10 hores

Casos d'ús coberts: Transversal.

7.3 Diagrama de Gantt

El Gantt complet és al projecte de ClickUp:

app.clickup.com/90152331687/v/s/90159849084



7.4 Rols de l'equip

Membre	Responsabilitats
Mohamed Amin Adaardour	**Infraestructura de xarxa, túnel d'accés extern, scripts d'instal·lació, proxy i traducció de protocols.**
Diego Morazán	**Web PHP, integració amb el sistema de gestió, documentació tècnica, repositori GitHub.**

8. Desenvolupament i implementació

8.1 Estructura del repositori

El projecte PiBlock s'ha organitzat amb una estructura mínima i plana que separa la instal·lació, les configuracions dels serveis i el panell web.

El repositori té quatre arrels (README.md, install.sh, config/ i web/):

- README.md — Descripció general i instruccions d'arrencada.
- install.sh — Script bash d'instal·lació automàtica de tot l'stack.

Descarrega Paper, Velocity, Geyser i Limbo, configura les unitats systemd, el firewall UFW, mDNS (avahi-daemon), Nginx amb PHP-FPM i fa un backup inicial.

- config/
  Configuracions dels serveis Minecraft en quatre subdirectoris (paper/, velocity/, geyser/, limbo/) amb les seves opcions i plugins/floodgate/. L'install.sh les descarrega a cada nova instal·lació.
- web/
- Panell web PHP servit per Nginx + PHP-FPM, amb tots els fitxers al mateix nivell:
- login.php — Inici de sessió
- register.php — Registre amb codis d'invitació
- dashboard.php — Panell principal
- config.php — Edita dificultat, mode i whitelist
- admin_users.php · delete_users.php — Gestió d'usuaris (només rol admin)

- logs.php — Registre d'activitat (només rol admin)
  auth.php — Guard de sessió · db.php — Connexió PDO a MariaDB · logout.php · style.css

- Estructura final
  El repositori és deliberadament petit i pla: l'install.sh és l'únic punt d'entrada, les configuracions viuen a config/ i el panell PHP a web/.

Aquesta estructura facilita la separació entre instal·lació, configuració dels serveis i panell web.

8.2 Bones pràctiques Git

Durant el desenvolupament del projecte s'ha utilitzat Git i GitHub per controlar versions i mantenir còpies de seguretat del codi.

El repositori principal del projecte és:

https://github.com/uhcv/PiBlock

Bones pràctiques aplicades

Cada canvi important del projecte s'ha guardar utilitzant commits per mantenir un historial dels canvis realitzats

Ús de commits descriptius

S'ha utilitzat missatges descriptius per identificar fàcilment les modificacions realitzades.

Repositori remot GitHub

El projecte es sincronitza amb GitHub mitjançant:

git push origin main

Això permet:

- Guardar còpies de seguretat.
- Compartir el projecte.
- Controlar canvis.
- Treballar de forma organitzada.

Organització del codi

Els fitxers s'han separat segons la seva funció:

- HTML/CSS frontend.
- PHP backend.
- Configuracions Linux.
- Scripts.
- Documentació.

Aquestes pràctiques permeten mantenir el projecte més ordenat i fàcil de mantenir.

~~Commits atòmics amb missatges en català i prefix de mòdul, com per exemple "[SRV] Configura Velocity per failover automàtic". La branca main és protegida; els canvis arriben per pull request amb revisió de l'altre membre. El .gitignore exclou claus, .env i fitxers de regió de Minecraft. Cada lliurament o fita rellevant té una etiqueta v1.0, v1.1, etc.~~

8.3 Tasca obligatòria: web PHP

La tasca obligatòria del mòdul M12 consisteix en desenvolupar una web PHP funcional amb autenticació, connexió a una base de dades i gestió del sistema. En el projecte PiBlock aquesta web fa de panell de control del servidor.

La web s'ha desenvolupat amb PHP 8, MariaDB i Bootstrap. El servidor web utilitzat és Nginx sobre Ubuntu Server. La interfície permet que un administrador pugui iniciar sessió, consultar l'estat del servidor i gestionar usuaris des del navegador.

El sistema funciona amb sessions PHP. Quan un usuari inicia sessió correctament, el servidor crea una sessió segura i restringeix l'accés a les pàgines internes si no hi ha autenticació.

La web està formada principalment per les següents pantalles:

- login.php →formulari d'inici de sessió
- register.php →registre de nous usuaris.
- dashboard.php →panell principal de control.
- config.php → configuració bàsica del servidor
- admin_users.php →gestió d'usuaris.
- logs.php →visualització dels registres del sistema

La base de dades MariaDB guarda els usuaris i els registres del sistema. Les contrasenyes no s'emmagatzemen en text pla, sinó amb hash bcrypt mitjançant password_hash().

Per evitar injeccions SQL s'utilitza PDO amb prepared statements a totes les consultes.

Exemple d'autenticació segura:

```
$stmt = $pdo->prepare("
SELECT id, username, password_hash, rol
FROM usuaris
WHERE username = :user
");

$stmt->execute([
    ':user' => $username
]);

$userData = $stmt->fetch(PDO::FETCH_ASSOC);

if ($userData &&
    password_verify($password, $userData['password_hash'])) {

    $_SESSION['user_id'] = $userData['id'];
    $_SESSION['username'] = $userData['username'];
    $_SESSION['rol'] = $userData['rol'];
}
```

La web també implementa control de rols. Els usuaris amb rol admin poden crear o eliminar comptes des del panell d'administració. Els accessos i tancaments de sessió queden registrats a la taula logs, i el registre s'autoritza amb codis d'invitació de la taula authorized_users de MariaDB.

El dashboard mostra informació del sistema i l'estat del servidor. La comprovació de l'estat es fa intentant connectar al servei mitjançant PHP, mostrant Online o Offline segons la resposta del servidor.

La part visual s'ha desenvolupat amb Bootstrap 5 i CSS personalitzat per mantenir una interfície moderna però lleugera. Tot el codi PHP es troba dins del directori web/ del repositori.

~~Esquema de la base de dades~~

~~Taula~~	~~Camps principals~~	~~Descripció~~
~~usuaris~~	~~id, username, password_hash, rol (admin/professor/user)~~	~~Usuaris de la web.~~
~~config_servidor~~	~~clau, valor~~	~~Configuració dinàmica del joc (dificultat, gamemode, whitelist).~~

8.4 Tasques opcionals completades

A més de les tasques obligatòries, durant el desenvolupament del projecte s'han implementat diverses funcionalitats addicionals per millorar el sistema.

Sistema de registre d'usuaris

S'ha desenvolupat una pàgina registre.php que permet crear nous comptes d'usuari directament des de la web.

Les contrasenyes s'emmagatzemen xifrades amb bcrypt utilitzant:

password_hash()

Sistema de rols

El sistema implementa control de rols:
- admin
- professor
- user

Els administradors poden:
- Gestionar usuaris.
- Eliminar comptes.
- Veure logs.
- Accedir a configuracions del sistema.

Sistema de logs

S'ha implementat un registre d'activitat que guarda:
- Inicis de sessió.
- Tancaments de sessió
- Accions del sistema

Els logs es guarden a MariaDB i es poden consultar des de la web.

Dashboard interactiu

S'ha desenvolupat un dashboard amb Bootstrap que mostra:
- Estat del servidor.
- Informació de l'usuari.
- Accessos ràpids.
- Configuració del sistema.

També s'ha implementat una comprovació d'estat del servidor mitjançant PHP per mostrar si els serveis estan Online o Offline

Integració amb Pyrodactyl

La pàgina principal integra un accés directe al panell Pyrodactyl utilitzat per administrar servidors de videojocs.

Adaptació Linux Ubuntu Server

El sistema PHP s'ha migrat des de Windows XAMPP cap a Ubuntu Server amb:

- Nginx.
- PHP 8.
- MariaDB.

També s'ha exportat la base de dades entre sistemes operatius.

Millora visual amb Bootstrap

La interfície s'ha millorat utilitzant:

- Bootstrap 5.
- Bootstrap Icons.
- CSS personalitzat.
- Disseny responsive.

L'objectiu ha estat crear una interfície moderna però senzilla, adequada per a un projecte SMX.

- ~~Preparació del maquinari: instal·lació, xarxa, hardening.~~
- ~~Script d'instal·lació automatitzada (bash, install.sh) que ho munta tot en una sola execució.~~
- ~~Configuració de la xarxa per a l'accés extern: firewall UFW obrint només els ports necessaris (22, 80, 443, 25565/TCP, 19132/UDP, 8080). Túnel Playit.gg com a pas manual segons l'entorn.~~
- ~~Stack de Minecraft amb fitxers documentats i optimitzats per a un ús escolar.~~

9. Desplegament

9.1 Arquitectura de desplegament

El sistema viu en una sola màquina: la Raspberry Pi 5. La Pi porta Paper, Velocity amb Geyser i Floodgate, i Limbo com a serveis systemd directes; el daemon Elytra de Pyrodactyl per a la gestió de servidors addicionals (que sí utilitza Docker); i la web PHP amb MariaDB servida per Nginx + PHP-FPM. A la xarxa local, avahi-daemon (mDNS) resol piblock.cat cap a la Pi. Per a l'accés extern, la configuració base obre els ports estrictament necessaris via UFW; opcionalment, el desplegament pot afegir un túnel Playit.gg per evitar tocar el router del centre.

9.2 Passos del desplegament

El procés en quatre blocs:

- Sistema base (1): Ubuntu Server al Pi 5, OpenJDK 21, Nginx + PHP-FPM, MariaDB, Docker (per al daemon Pyrodactyl) i UFW configurat.
- Sistema de gestió (2): el daemon Elytra de Pyrodactyl s'instal·la com a unitat systemd al mateix Pi 5. La configuració d'Elytra contra un panell Pyrodactyl extern (elytra configure --panel-url <URL> --token <TOKEN> --node <ID>) es deixa com a pas manual posterior a la instal·lació.
- Stack del joc (3): serveis systemd piblock-paper, piblock-velocity, piblock-geyser i piblock-limbo configurats per al failover a Limbo. Configuracions descarregades des de config/ del repositori.

- Web i accés (4): web PHP desplegada a /var/www/piblock amb MariaDB a la mateixa Pi; opcionalment, configuració manual d'un túnel Playit.gg per a l'accés extern.

9.3 Incidències trobades

- Incompatibilitat Geyser-Velocity: la versió inicial de Geyser no era compatible. Resolt fixant les URLs de descàrrega a versions concretes a l'install.sh.
- Permisos de Docker: l'usuari admin no podia executar Docker sense sudo. Resolt afegint l'usuari al grup docker.
- Latència alta per DNS: connexions externes a > 300 ms per la resolució del nom local. Resolt configurant avahi-daemon (mDNS) per anunciar piblock i piblock.cat a la xarxa local.
- Aturada bruta del servidor: Paper no rebia bé el senyal d'aturada. Resolt configurant Restart=on-failure i RestartSec=10 a la unitat systemd piblock-paper.service.

9.4 Validació final

Després del desplegament hem validat que tots els casos d'ús principals funcionen. El detall és a la secció 10. El sistema ha quedat operatiu amb les funcionalitats bàsiques i tres pendents d'acabar (taula d'objectius de la secció 10.1).

10. Resultats i validació

10.1 Objectius assolits

Objectiu	Estat	CU
Servidor accessible a Java i Bedrock	**✓ Completat**	**CU-05**
Accés extern sense obrir ports	**✓ Completat**	**CU-05, CU-06**
Sala d'espera automàtica si cau el servidor	**✓ Completat**	**CU-05**
Gestió remota per a l'admin	**✓ Completat**	**CU-01, CU-02**
Registre de noves màquines	**⚙ En curs**	**CU-03, CU-04**
Còpies de seguretat programades	**⚙ En curs**	**CU-02**

Configuració i supervisió pel professor	**⚙ En curs**	**CU-07, CU-08**

10.2 Proves realitzades

Hem fet 19 proves cobrint tots els casos d'ús i la web PHP. Totes han passat. La taula següent mostra les més rellevants; la llista completa és al repositori del projecte.

Cas de prova	Resultat esperat	Obtingut	Estat
Connexió Java (CU-05)	**Jugador entra al món**	**OK**	✓
Connexió Bedrock (CU-05)	**Jugador entra (traducció transparent)**	**OK, veu jugadors Java**	✓
Failover sala espera (CU-05)	**Redirecció < 5s i tornada automàtica**	**Redirecció en 1s, tornada en 1s**	✓
Accés extern (CU-05)	**Latència < 120 ms**	**0ms (va ser prova local)**	-
Còpia manual (CU-02)	**Còpia creada en segon pla**	**145 MB en 8 s**	✓
Restauració (CU-02)	**Món restaurat, servidor reiniciat**	**Estat correcte**	✓
Injecció SQL (web)	**Login rebutjat sense error**	**Rebutjat**	✓
Càrrega 8 jugadors	**Latència < 120 ms, RAM < 4 GB**	**85 ms / 3.2 GB**	✓

10.3 Feedback i validació externa

El tutor ens ha anat corregint al llarg dels tres trimestres. Tres observacions principals: reduir l'abast en lloc de voler-ho cobrir tot, justificar les decisions tècniques mostrant alternatives descartades (d'aquí la taula de la 6.5) i assegurar que la descomposició en capes sigui completa i coherent.

Companys de classe han provat el servidor en sessions de classe i ens han donat opinions sobre l'experiència. Les proves de càrrega amb diferents dispositius i versions del client venen d'aquí.

11. Conclusions i aprenentatges

11.1 El que ha funcionat

La hipòtesi era que un dispositiu compacte pot fer de servidor escolar si tries bé els components. El sistema aguanta una aula sencera (al voltant de 30 jugadors simultanis) amb latència bona, i la combinació de proxy + traducció + servidor optimitzat respon.

La part més satisfactòria és el túnel d'accés extern via Playit.gg. Quan vas a un centre i et pregunten "haurem d'obrir ports al router?" i pots dir "no", és un moment que paga la pena.

Derivar els components dels casos d'ús i la descomposició en capes ha funcionat: cap funcionalitat ha quedat penjada i cada component té una raó concreta.

11.2 El que ha costat més

La integració de Geyser amb Velocity va donar més guerra de la que esperàvem. La compatibilitat entre versions no és evident, i vam passar un parell de dies depurant un problema que era una incompatibilitat de versions mal documentada.

La part de PHP és la que ha quedat més incompleta. Integrar la llista d'autoritzats amb l'API del sistema de gestió és més complex del que semblava i no l'hem tancat. La configuració educativa pel professor (CU-07, CU-08) també queda pendent.

11.3 Millores futures

- Còpies automàtiques programades amb restauració des de la web.
- Completar la interfície educativa del professor (CU-07, CU-08).
- Automatitzar del tot l'alta de noves màquines.

11.4 Reflexió sobre els mòduls

M07 (Serveis de Xarxa) i M09 (Seguretat) han pesat molt: el primer per l'stack de Minecraft i el proxy, el segon pel túnel xifrat. M10 (Aplicacions Web) ha quedat curt perquè la web encara té coses pendents. M02 ha servit per la selecció del maquinari i M05 per la configuració del tallafoc i la segmentació de contenidors.

12. Annexos

Annex A. Repositori i recursos

- Repositori GitHub: github.com/uhcv/PiBlock
- Diagrama de Gantt (ClickUp): https://app.clickup.com/90152331687/v/s/90159849084
- Domini i web pública: http://piblock.cat
- Panell (BETA): http://piblock.cat
- Presentació: https://uhcv.github.io/

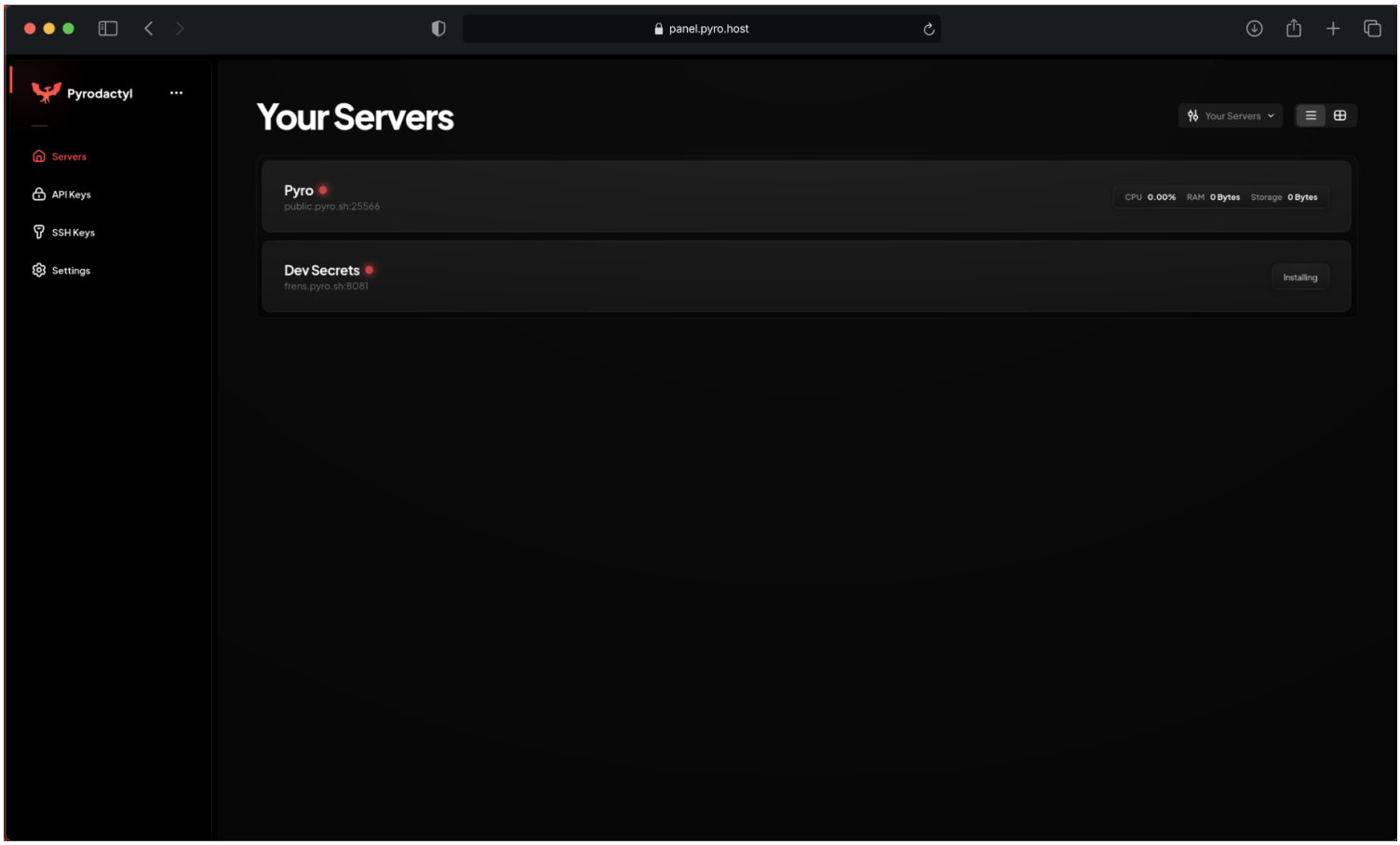
Annex B. Fragment de la configuració de Velocity

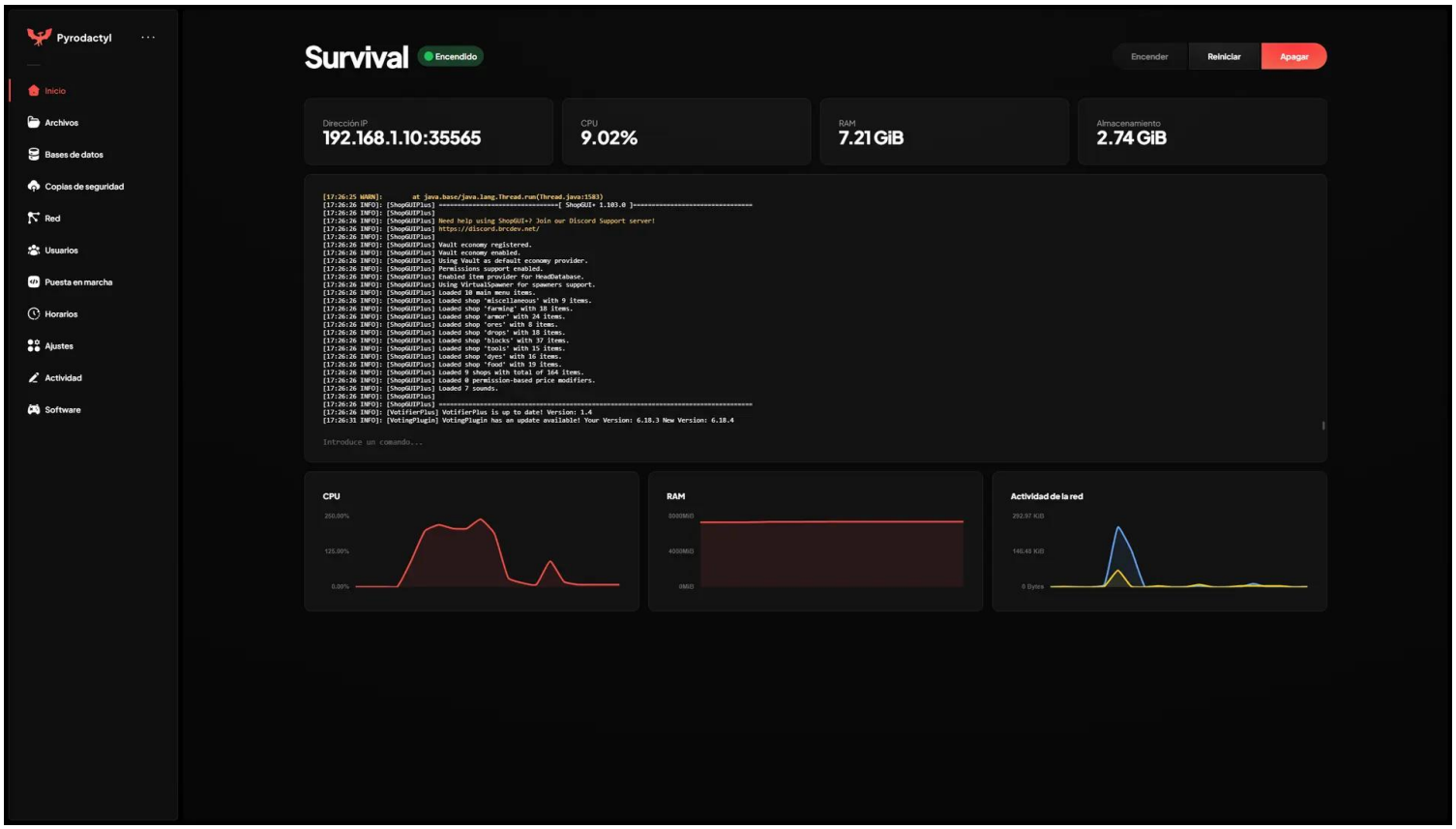
```
bind = "0.0.0.0:25565"
```

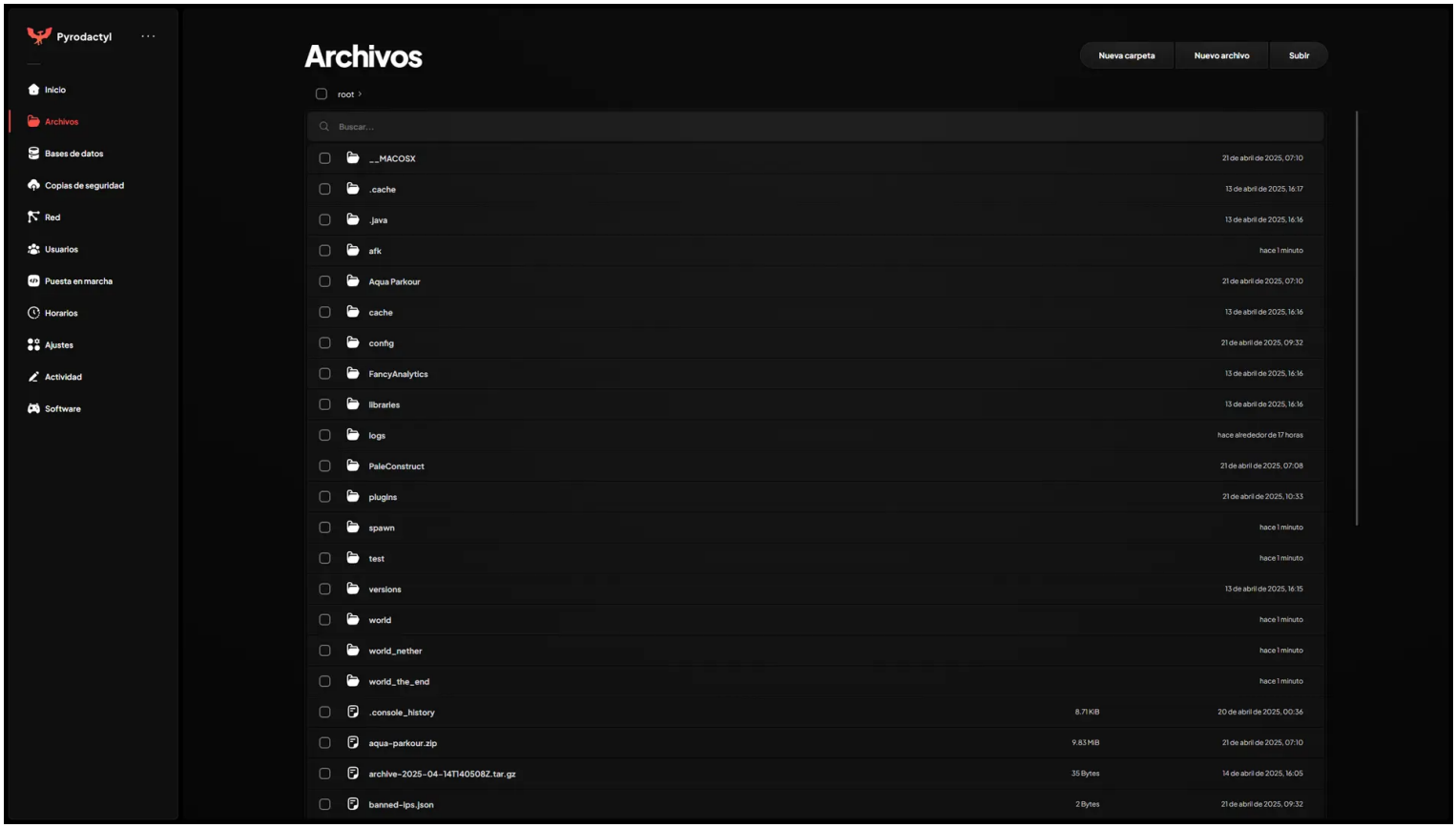
```
motd = "<#09add3>A Velocity Server"
show-max-players = 500
online-mode = true
player-info-forwarding-mode = "modern"

[servers]
lobby = "127.0.0.1:30066"
limbo = "127.0.0.1:30000"
try = [
    "lobby",
    "limbo"
]
```

La resta de fitxers (Paper, Geyser, Limbo, Docker Compose) són a /srv/ del repositori.

Annex C. Captures de pantalla


panel.pyro.host
Pyrodactyl
Servers
API Keys
SSH Keys
Settings
Your Servers
Your Servers
Pyro
public.pyro.sh:25566
CPU 0.00% RAM 0 Bytes Storage 0 Bytes
Dev Secrets
frens.pyro.sh:8081
Installing


Pyrodactyl
Inicio
Archivos
Bases de datos
Copias de seguridad
Red
Usuarios
Puesta en marcha
Horarios
Ajustes
Actividad
Software
Survival
Encendido
Encender
Reiniciar
Apagar
Dirección IP
192.168.1.10:35565
CPU
9.02%
RAM
7.21 GiB
Almacenamiento
2.74 GiB
[17:26:25 WARN]: at java.base/java.lang.Thread.run(Thread.java:1583)
[17:26:26 INFO]: [ShopGUIPlus] ==========[ShopGUI+ 1.103.0]==========
[17:26:26 INFO]: [ShopGUIPlus]
[17:26:26 INFO]: [ShopGUIPlus] Need help using ShopGUI+? Join our Discord Support server!
[17:26:26 INFO]: [ShopGUIPlus] https://discord.brcdev.net/
[17:26:26 INFO]: [ShopGUIPlus]
[17:26:26 INFO]: [ShopGUIPlus] Vault economy registered.
[17:26:26 INFO]: [ShopGUIPlus] Vault economy enabled.
[17:26:26 INFO]: [ShopGUIPlus] Using Vault as default economy provider.
[17:26:26 INFO]: [ShopGUIPlus] Permissions support enabled.
[17:26:26 INFO]: [ShopGUIPlus] Enabled item provider for HeadDatabase.
[17:26:26 INFO]: [ShopGUIPlus] Using VirtualSpawner for spawners support.
[17:26:26 INFO]: [ShopGUIPlus] Loaded 10 main menu items.
[17:26:26 INFO]: [ShopGUIPlus] Loaded shop 'miscellaneous' with 9 items.
[17:26:26 INFO]: [ShopGUIPlus] Loaded shop 'farming' with 18 items.
[17:26:26 INFO]: [ShopGUIPlus] Loaded shop 'armor' with 24 items.
[17:26:26 INFO]: [ShopGUIPlus] Loaded shop 'ores' with 8 items.
[17:26:26 INFO]: [ShopGUIPlus] Loaded shop 'drops' with 18 items.
[17:26:26 INFO]: [ShopGUIPlus] Loaded shop 'blocks' with 37 items.
[17:26:26 INFO]: [ShopGUIPlus] Loaded shop 'tools' with 15 items.
[17:26:26 INFO]: [ShopGUIPlus] Loaded shop 'dyes' with 16 items.
[17:26:26 INFO]: [ShopGUIPlus] Loaded shop 'food' with 19 items.
[17:26:26 INFO]: [ShopGUIPlus] Loaded 9 shops with total of 164 items.
[17:26:26 INFO]: [ShopGUIPlus] Loaded 0 permission-based price modifiers.
[17:26:26 INFO]: [ShopGUIPlus] Loaded 7 sounds.
[17:26:26 INFO]: [ShopGUIPlus]
[17:26:26 INFO]: [ShopGUIPlus] ==========
[17:26:26 INFO]: [VotifierPlus] VotifierPlus is up to date! Version: 1.4
[17:26:31 INFO]: [VotingPlugin] VotingPlugin has am update available! Your Version: 6.18.3 New Version: 6.18.4
Introduce un comando...
CPU
250.00%
125.00%
0.00%
RAM
8000MiB
4000MiB
0MiB
Actividad de la red
292.97 KiB
146.48 KiB
0 Bytes


Pyrodactyl
Inicio
Archivos
Bases de datos
Copias de seguridad
Red
Usuarios
Puesta en marcha
Horarios
Ajustes
Actividad
Software
Archivos
Nueva carpeta
Nuevo archivo
Subir
root ›
Buscar...
__MACOSX 21 de abril de 2025, 07:10
.cache 13 de abril de 2025, 16:17
.java 13 de abril de 2025, 16:16
afk hace 1 minuto
Aqua Parkour 21 de abril de 2025, 07:10
cache 13 de abril de 2025, 16:16
config 21 de abril de 2025, 09:32
FancyAnalytics 13 de abril de 2025, 16:16
libraries 13 de abril de 2025, 16:16
logs hace alrededor de 17 horas
PaleConstruct 21 de abril de 2025, 07:08
plugins 21 de abril de 2025, 10:33
spawn hace 1 minuto
test hace 1 minuto
versions 13 de abril de 2025, 16:15
world hace 1 minuto
world_nether hace 1 minuto
world_the_end hace 1 minuto
.console_history 8.71 KiB 20 de abril de 2025, 00:36
aqua-parkour.zip 9.83 MiB 21 de abril de 2025, 07:10
archive-2025-04-14T140508Z.tar.gz 35 Bytes 14 de abril de 2025, 16:05
banned-ips.json 2 Bytes 21 de abril de 2025, 09:32

- Captures del panell Pyrodactyl, de la web PHP de gestió i de la consola d'instal·lació -

Annex D. Scripts d'instal·lació

Per facilitar la posada en marxa d'una nova màquina PiBlock, hem creat un script d'instal·lació automàtica que descarrega i configura tot l'stack del joc, el panell Pyrodactyl i el túnel d'accés extern en una sola comanda. El procés detecta l'arquitectura del sistema, instal·la OpenJDK 21, descarrega Paper, Velocity, Geyser i Floodgate, configura Limbo com a sala d'espera, registra la màquina automàticament mitjançant Elytra, configura systemd i el firewall i fa un primer backup. La instal·lació completa triga uns 50 segons en una Raspberry Pi 5.

curl -sSL piblock.cat/install.sh **| sudo bash**

```
root@piblock:~# curl -sSL piblock.cat/install.sh | sudo bash
────────────────────────────────────────
▸ Detectant arquitectura ............... amd64
▸ Instal·lant OpenJDK ................ 18.4 s
▸ Descarregant Paper ................. 3.1 s
▸ Descarregant Velocity .............. 1.7 s
▸ Plugins · Geyser + Floodgate ...... 5.2 s
▸ Limbo (servidor d'espera) ......... 1.4 s
▸ Elytra (daemon Pyrodactyl) ........ 2.9 s
▸ Configurant systemd · firewall ..... 1.8 s
▸ Primer backup ...................... 0.8 s
────────────────────────────────────────
✓ A punt en 35 s.
Panell:  http://piblock.cat
Server: piblock.cat · 25565 (Java)
       piblock.cat · 19132 (Bedrock)

Per arrencar els servidors Minecraft:
  /opt/piblock/start_all.sh

⚠ Elytra està instal·lat però NO configurat.
  Per engegar-lo, has de seguir aquests passos:
  1. Afegeix un node al teu panell de Pyrodactyl
  2. Copia l'ordre auto-deploy (botó Configuration)
  3. Executa:  cd /etc/elytra && elytra configure --panel-url <URL> --token <TOKEN> --node <ID>
  4. Activa:    systemctl enable --now elytra
```